

Part A: Dataset Preparation

Task 1: Download and Prepare Dataset

Screenshot of the dataset file on the learner system

The screenshot shows the UCI Machine Learning Repository page for the 'Online Retail' dataset. The page is titled 'Online Retail' and was donated on 11/5/2015. It provides a description of the dataset as a transactional data set containing all transactions between 01/12/2010 and 09/12/2011 for a UK-based and registered non-store online retail. The page includes a table of dataset characteristics, subject area, associated tasks, feature type, number of instances, and number of features. It also has buttons for downloading the dataset (22.6 MB), importing it into Python, and citing it. The page lists 8 citations and 552362 views. The keywords section includes 'sales'. The creators section lists Daqing Chen, chend@lsbu.ac.uk, School of Engineering, London South Bank University.

Dataset Characteristics	Subject Area	Associated Tasks
Multivariate, Sequential, Time-Series	Business	Classification, Clustering

Feature Type	# Instances	# Features
Integer, Real	541909	6

Dataset Information

Additional Information

This is a transactional data set which contains all the transactions occurring between 01/12/2010 and 09/12/2011 for a UK-based and registered non-store online retail. The company mainly sells unique all-occasion gifts. Many customers of the company are wholesalers.

Has Missing Values?

Screenshot of the README file

The screenshot shows a Notepad window titled 'readme.txt - Notepad'. The text inside the window reads:

```
File Edit Format View Help
Dataset Name: Online Retail Dataset
Source: UCI Machine Learning Repository
File Name: online_retail.csv
Purpose: AWS S3, Databricks, and Delta Lake assignment
```

Final dataset file: online_retail.csv

The screenshot shows a file explorer window with the following path: USB Drive (D:) > Raj1 > HeroVired > Graded Assignment > 05_AWS Online Retail Delta Lake > Assignment_Spark on Cloud + Delta Lake > Dataset_OnlineRetail. The file explorer shows two files: 'Online Retail.csv' (45,052 KB) and 'Online Retail.xlsx' (23,160 KB).

Part B: AWS Cost Monitoring

Task 2: Create AWS Budget Alert

Create AWS Budget Alert

Choose budget type

Budget setup

☒ Use a template (default)
Use the template that best matches your use case.

☐ Custom (advanced)
Customize a budget to fit your specific needs. You can customize the time period, the time range, and create custom rules.

Billing View - optional
Select a billing view you want to create this budget for. This will not be editable after creation. If no view is selected, the report will be created for your account data (primary view).

Templates - new
Choose a template that best matches your use case.

☒ 2023 spend budget
Create a budget that tracks your 2023 spending across all regions.

☐ Monthly cost budget
Create a monthly budget that notifies you if you spend or are forecasted to exceed the budget amount.

☐ Daily Savings Plans coverage budget
Create a coverage budget for your Savings Plans that notifies you when you fall below the defined target.

☐ Daily reservation utilization budget
Create a utilization budget for your reservation instances that notifies you when you fall below the defined target.

Zero spend budget - Template

Budget name
Provide a descriptive name for this budget.
monthly-retail-project-budget

Email recipients
Select one or more email addresses to notify when the budget is exceeded.
raj.kumar@raj20kumar.com

Notes
Add a note to this budget (optional).

Scope
All AWS services are in scope in this budget.

☒ You will be notified via email when any spend above \$101 is forecasted.

Template settings
This template has default configurations that can be changed later to change any of these settings, see [Customize](#). You can also download this template as JSON.

[Cancel](#) [Create budget](#)

Task 3: Open AWS Cost Explorer

Budgets (1)

[Download CSV](#) [Actions](#) [Create budget](#)

Name	Thresholds	Health status	Billing view	Budget	Amount	Forecast	Current vs. budgeted	Forecasted vs. bud...
monthly-retail-project-budget	OK	Healthy	Primary View	\$1.00	-	-	0.00%	-

Part C: AWS S3 Implementation

Task 4: Create S3 Bucket

Buckets

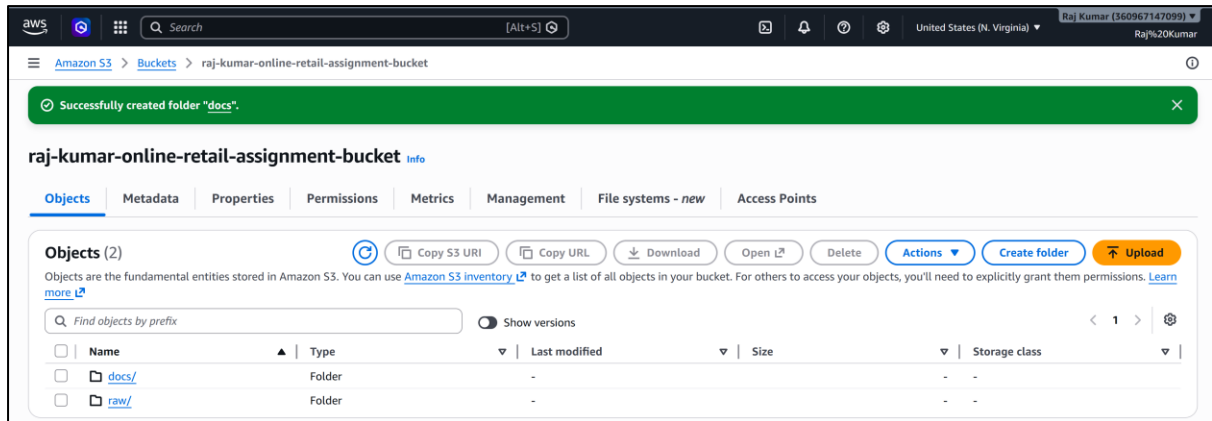
General purpose buckets (3)

Buckets are containers for data stored in S3.

Name	AWS Region	Creation date
databricks-s3-ingest-eel10-lambda2ipbucket-ksvgopsgpt	US East (N. Virginia) us-east-1	June 8, 2026, 18:07:50 (UTC+05:30)
<u>raj-kumar-online-retail-assignment-bucket</u>	US East (N. Virginia) us-east-1	June 10, 2026, 17:29:49 (UTC+05:30)
sportsbar-ri1729	Europe (Stockholm) eu-north-1	June 8, 2026, 17:30:47 (UTC+05:30)

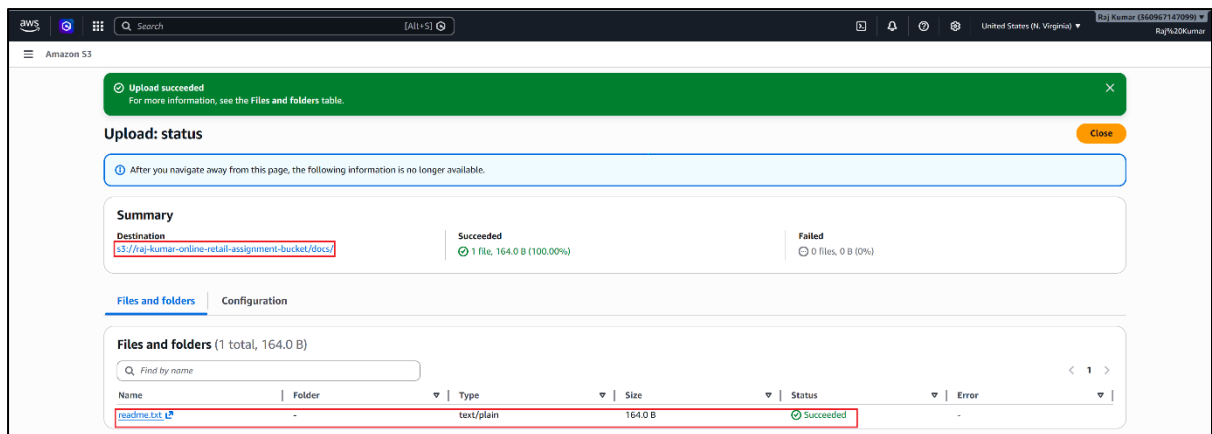
Task 5: Create Folder Structure in S3

folders inside the S3 bucket

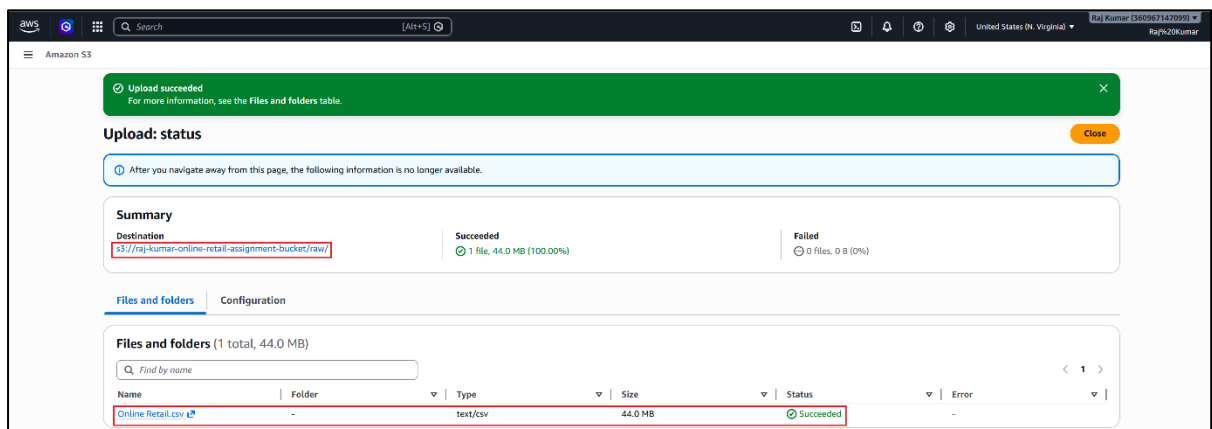


Task 6: Upload Dataset and README to S3

readme.txt uploaded inside /docs/



online_retail.csv uploaded inside /raw/



Task 7: Configure S3 Lifecycle Rule

Amazon S3

Buckets

raj-kumar-online-retail-assignment-bucket

Lifecycle configuration

Create lifecycle rule

Create lifecycle rule

Lifecycle rule configuration

Lifecycle rule name

online-retail-lifecycle-rule

Up to 255 characters

Choose a rule scope

☒ Limit the scope of this rule using one or more filters

☐ Apply to all objects in the bucket

Filter type

You can filter objects by prefix, object tags, object size, or whatever combination suits your use case.

Prefix

Add filter to limit the scope of this rule to a single prefix.

row/

Don't include the bucket name in the prefix. Using certain characters in key names can cause problems with some applications and protocols. [Learn more](#)

Object tags

You can limit the scope of this rule to the key/value pairs added below.

Add tag

Object size

You can limit the scope of this rule to apply to objects based on their size. [Learn more](#)

☐ Specify minimum object size

☐ Specify maximum object size

Lifecycle rule actions

Choose the actions you want this rule to perform.

☒ Transition current versions of objects between storage classes

This action will move current versions.

☐ Transition noncurrent versions of objects between storage classes

This action will move noncurrent versions.

☐ Expire current versions of objects

This action will remove noncurrent versions.

☐ Expire current versions of objects

This action will remove noncurrent versions of objects.

Amazon S3

Buckets

raj-kumar-online-retail-assignment-bucket

Lifecycle configuration

Create lifecycle rule

Lifecycle rule actions

Choose the actions you want this rule to perform.

☒ Transition current versions of objects between storage classes

This action will move current versions.

☐ Transition noncurrent versions of objects between storage classes

This action will move noncurrent versions.

☐ Expire current versions of objects

This action will remove noncurrent versions.

☐ Permanently delete noncurrent versions of objects

☒ Delete expired object delete markers or incomplete multipart uploads

These actions are not supported when filtering by object tags or object size.

Transitions are charged per request

For a lifecycle transition action, each request corresponds to an object transition. For details on lifecycle transition pricing, see requests pricing info on the [Storage & requests tab](#) of the [Amazon S3 pricing page](#).

☒ I acknowledge that this lifecycle rule will incur a transition cost per request.

By default, objects less than 128KB will not transition across any storage class

We don't recommend transitioning objects less than 128 KB because the transition costs can outweigh the storage savings. If your use case requires transitioning objects less than 128 KB, specify a minimum object size filter for each applicable lifecycle rule with a transition action.

Transition current versions of objects between storage classes

Choose transitions to move current versions of objects between storage classes based on your use case scenario and performance access requirements. These transitions start from when the objects are created and are consecutively applied. [Learn more](#)

Choose storage class transitions

Standard-IA

Days after object creation

90

Remove

Add transition

Delete expired object delete markers or incomplete multipart uploads

Expired object delete markers

This action will remove expired object delete markers and may improve performance. An expired object delete marker is removed if all noncurrent versions of an object expire after deleting a noncurrent object. This action is not available when "Expire current versions of objects" is selected. [Learn more](#)

☒ Delete expired object delete markers

Incomplete multipart uploads

This action will stop all incomplete multipart uploads, and the parts associated with the multipart upload will be deleted. [Learn more](#)

☒ Delete incomplete multipart uploads

Number of days

90

Integer must be greater than 0.

Amazon S3

Buckets

raj-kumar-online-retail-assignment-bucket

Lifecycle configuration

Create lifecycle rule

Transition current versions of objects between storage classes

Choose transitions to move current versions of objects between storage classes based on your use case scenario and performance access requirements. These transitions start from when the objects are created and are consecutively applied. [Learn more](#)

Choose storage class transitions

Standard-IA

Days after object creation

90

Remove

Add transition

Delete expired object delete markers or incomplete multipart uploads

Expired object delete markers

This action will remove expired object delete markers and may improve performance. An expired object delete marker is removed if all noncurrent versions of an object expire after deleting a noncurrent object. This action is not available when "Expire current versions of objects" is selected. [Learn more](#)

☒ Delete expired object delete markers

Incomplete multipart uploads

This action will stop all incomplete multipart uploads, and the parts associated with the multipart upload will be deleted. [Learn more](#)

☒ Delete incomplete multipart uploads

Number of days

90

Integer must be greater than 0.

Review transition and expiration actions

Current version actions

Day 0

• Objects uploaded

↓

Day 30

• Objects move to Standard-IA

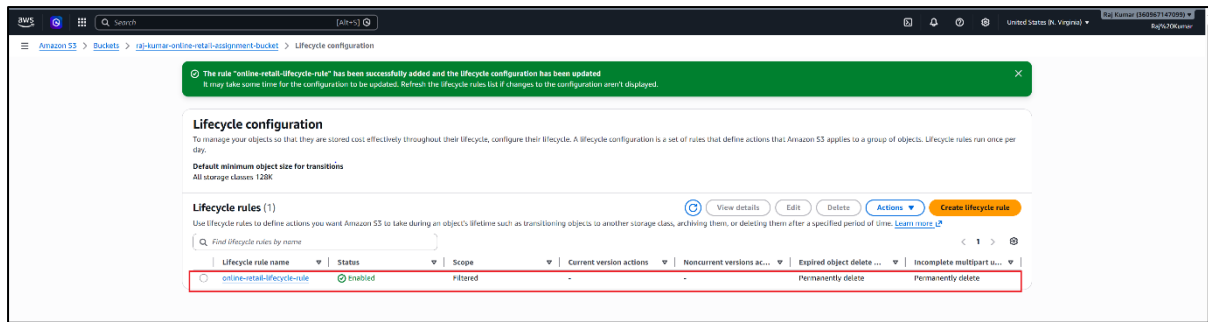
Noncurrent versions actions

Day 0

No actions defined.

Cancel

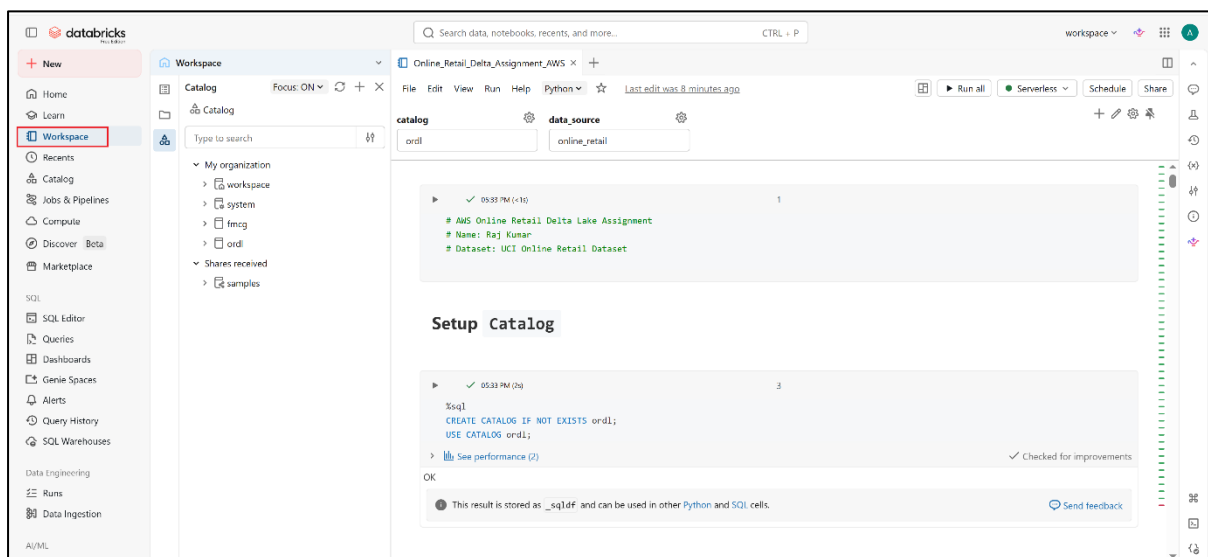
Create rule



Part D: Databricks Setup

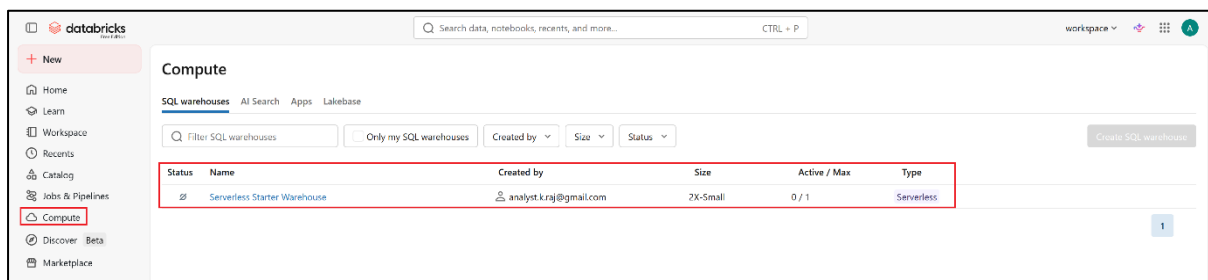
Task 8: Access Databricks Workspace

Screenshot of the Databricks workspace home page

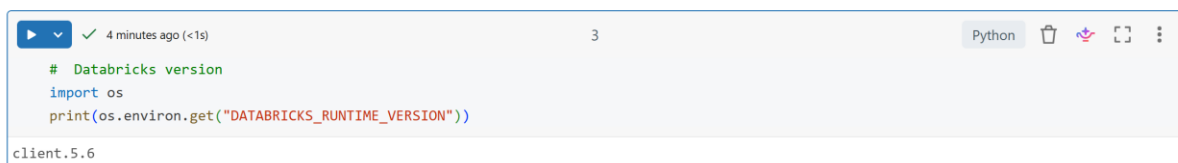


Task 9: Create Databricks Cluster

Cluster name



Runtime version.



Cluster status as running.

[SQL Warehouses](#) >

Serverless Starter Warehouse •

Overview

Connection details

Monitoring

Status

● Running

Name

Serverless Starter Warehouse (ID: 7e0f4de01f7c0453)

Type

Serverless

Cluster size

2X-Small

Auto stop

After 10 minutes of inactivity

Scaling

• Cluster count: Active 1 Min 1 Max 1

Channel

Current (v 2026.10)

Created by

analyst.k.raj@gmail.com

Cluster configuration page.

databricks
Free Edition

+ New

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover Beta

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Spaces

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

AI/ML

Platform

Search data, notebooks, recents, and more...

Serverless Starter Warehouse •

Overview

Connection details

Monitoring

Use these details to connect to this warehouse

Tableau

Power BI

dbt

Python

Java

Node.js

Go

More tools

Server hostname

dbc-78f69b6c-3370.cloud.databricks.com

Workspace ID

7474649799398365

HTTP path

/sql/1.0/warehouses/7e0f4de01f7c0453

JDBC URL

2.6.25 or later

jdbc:databricks://dbc-78f69b6c-3370.cloud.databricks.com:443/default;transportMode=http;ssl=1;AuthMech=3;httpPath=/sql/1.0/warehouses/7e0f4de01f7c0453;

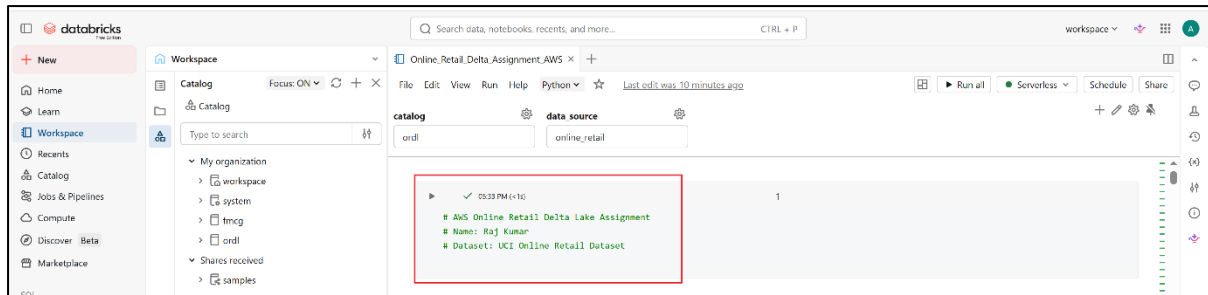
Databricks supports drivers released within the last two years. [Download drivers here](#)

OAuth URL

https://dbc-78f69b6c-3370.cloud.databricks.com/oidc

Task 10: Create Databricks Notebook

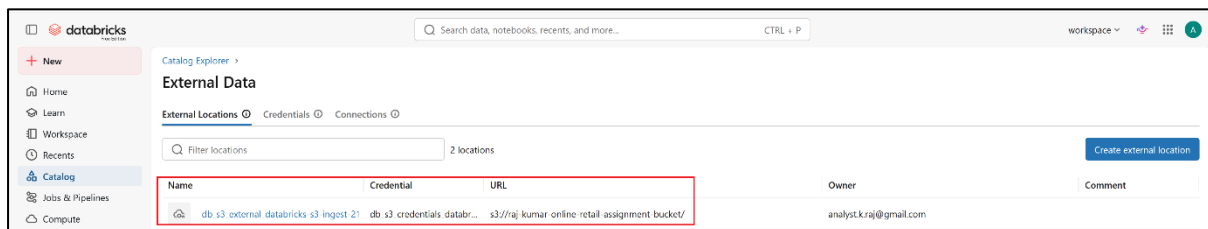
Screenshot of the first notebook cell.



Part E: Load Dataset in Databricks

Task 11: Upload Dataset to DBFS

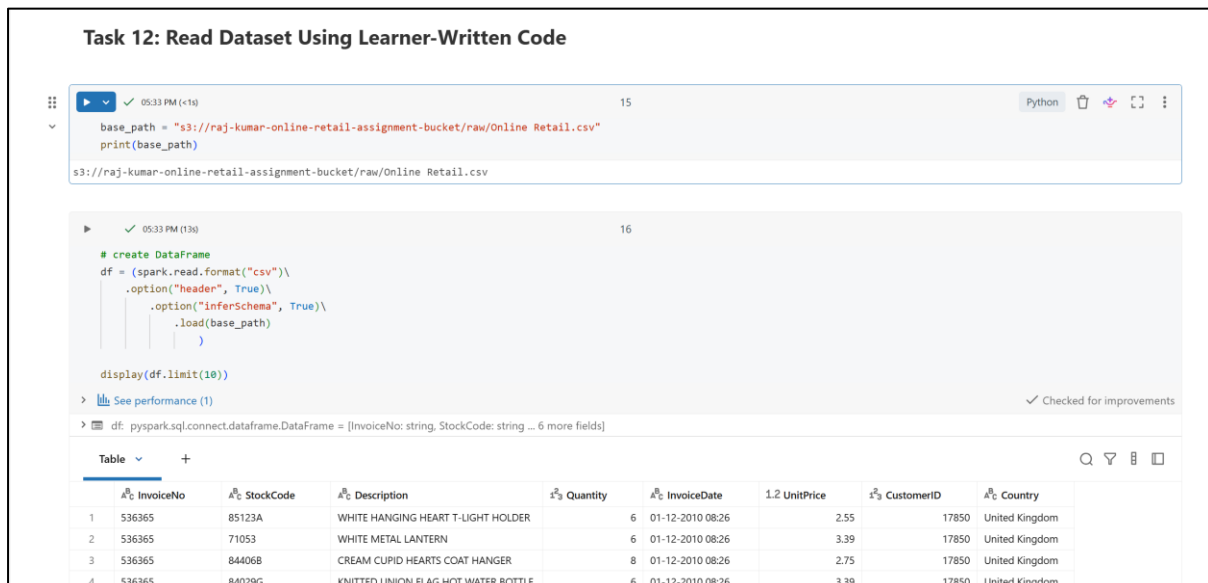
Screenshot showing the uploaded dataset in DBFS.



Task 12: Read Dataset Using Learner-Written Code

Learner-written code.

Dataset preview.



Schema output.

Row count output.

```
05:33 PM (<1s) 17
# print check data type
df.printSchema()

root
 |-- InvoiceNo: string (nullable = true)
 |-- StockCode: string (nullable = true)
 |-- Description: string (nullable = true)
 |-- Quantity: integer (nullable = true)
 |-- InvoiceDate: string (nullable = true)
 |-- UnitPrice: double (nullable = true)
 |-- CustomerID: integer (nullable = true)
 |-- Country: string (nullable = true)

05:33 PM (3s) 18
print(f'total row count: {df.count()}')
> See performance (1) ✓ Checked for improvements
total row count: 541909
```

Part F: Create Bronze Delta Table

Task 13: Create Bronze Delta Table Using Learner-Written Code

Learner-written Bronze table code.

Successful table creation.

Bronze table preview.

Task 13: Create Bronze Delta Table Using Learner-Written Code

```
05:33 PM (21s) 20
# write data in bronze layer
df.write\
    .format("delta")\
    .option("delta.enableChangeDataFeed", True)\
    .mode("overwrite")\
    .saveAsTable(f'{catalog}.{bronze_schema}.{data_source}')
> See performance (1) ✓ Checked for improvements
```

```
05:33 PM (3s) 21
%sql
SELECT * FROM ord1.bronze.online_retail
LIMIT 20;
> See performance (1) ✓ Checked for improvements
```

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
1	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	01-12-2010 08:26	2.55	17850	United Kingdom
2	536365	71053	WHITE METAL LANTERN	6	01-12-2010 08:26	3.39	17850	United Kingdom
3	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	01-12-2010 08:26	2.75	17850	United Kingdom
4	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	01-12-2010 08:26	3.39	17850	United Kingdom
5	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	01-12-2010 08:26	3.39	17850	United Kingdom
6	536365	22752	SET 7 BABUSHKA NESTING BOXES	2	01-12-2010 08:26	7.65	17850	United Kingdom

Task 14: Clean and Transform the Dataset Using Learner-Written Code

Learner-written transformation code.

Task 14: Clean and Transform the Dataset Using Learner-Written Code

05:33 PM (1s) 23

```
# • Read data from the Bronze Delta table
bronze_df = spark.read.format('delta').table('ord1.bronze.online_retail')
```

bronze_df: pyspark.sql.connect.dataframe.DataFrame = [InvoiceNo: string, StockCode: string ... 6 more fields]

05:33 PM (1s) 24

```
bronze_df.count()
```

[See performance \(1\)](#) ✓ Checked for improvements

541909

05:33 PM (<1s) 25

```
# • Remove duplicate records.
clean_df = bronze_df.dropDuplicates()
```

clean_df: pyspark.sql.connect.dataframe.DataFrame = [InvoiceNo: string, StockCode: string ... 6 more fields]

05:33 PM (1s) 26

```
clean_df.count()
```

[See performance \(1\)](#) ✓ Checked for improvements

536641

05:33 PM (2s) 27

```
# • Drop rows where InvoiceNo, StockCode, or Description is missing.
clean_df = clean_df.dropna(subset=['InvoiceNo', 'StockCode', 'Description'])
clean_df.count()
```

[See performance \(1\)](#) [Optimize](#)

clean_df: pyspark.sql.connect.dataframe.DataFrame = [InvoiceNo: string, StockCode: string ... 6 more fields]

535187

05:33 PM (1s) 28

```
clean_df.printSchema()
clean_df.show(10)
```

[See performance \(1\)](#) [Optimize](#)

```
root
|-- InvoiceNo: string (nullable = true)
|-- StockCode: string (nullable = true)
|-- Description: string (nullable = true)
|-- Quantity: integer (nullable = true)
|-- InvoiceDate: string (nullable = true)
|-- UnitPrice: double (nullable = true)
|-- CustomerID: integer (nullable = true)
|-- Country: string (nullable = true)

+-----+-----+-----+-----+-----+-----+
|InvoiceNo|StockCode|      Description|Quantity|  InvoiceDate|UnitPrice|CustomerID|  Country|
+-----+-----+-----+-----+-----+-----+
|  536365|  85123A|WHITE HANGING HEA...|        6|6|01-12-2010 08:26|    2.55|   17850|United Kingdom|
|  536365|  71053|WHITE METAL LANTERN|        6|6|01-12-2010 08:26|    3.39|   17850|United Kingdom|
|  536365|  84406B|CREAM CUPID HEART...|        8|8|01-12-2010 08:26|    2.75|   17850|United Kingdom|
```

- Silver Delta table output.

05:33 PM (5s) 39

```
# • Save the cleaned data in Delta format at /delta/silver/online_retail.
clean_df.write\
  .format("delta")\
  .option("overwriteSchema", True)\
  .mode("overwrite")\
  .saveAsTable(f'{catalog}.{silver_schema}.{data_source}')
```

[See performance \(1\)](#) [Optimize](#)

05:33 PM (2s) 40

```
# • Read data from the silver Delta table
# • Preview at least 20 records from the Silver table.
silver_df = spark.read.format("delta").table(f'{catalog}.{silver_schema}.{data_source}')
display(silver_df.limit(20))
```

[See performance \(1\)](#) [Checked for improvements](#)

silver_df: pyspark.sql.connect.dataframe.DataFrame = [InvoiceNo: string, StockCode: string ... 10 more fields]

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	Inv
2	536373	22752	SET 7 BABUSHKA NESTING BOXES	2	2010-12-01T09:02:00.000+00:00	7.65	17850	UNITED KINGDOM	2010-12
3	536375	37370	RETRO COFFEE MUGS ASSORTED	6	2010-12-01T09:32:00.000+00:00	1.06	17850	UNITED KINGDOM	2010-12
4	536380	22961	JAM MAKING SET PRINTED	24	2010-12-01T09:41:00.000+00:00	1.45	17809	UNITED KINGDOM	2010-12
5	536381	37444C	PINK BREAKFAST CUP AND SAUCER	1	2010-12-01T09:41:00.000+00:00	2.95	15311	UNITED KINGDOM	2010-12

- Final schema.

05:33 PM (<1s)

```
# • Display the final Silver table schema.
silver_df.printSchema()
```

```
root
|-- InvoiceNo: string (nullable = true)
|-- StockCode: string (nullable = true)
|-- Description: string (nullable = true)
|-- Quantity: integer (nullable = true)
|-- InvoiceDate: timestamp (nullable = true)
|-- UnitPrice: double (nullable = true)
|-- CustomerID: integer (nullable = true)
|-- Country: string (nullable = true)
|-- invoice_date_only: date (nullable = true)
|-- invoice_month: string (nullable = true)
|-- total_amount: double (nullable = true)
|-- is_cancelled_invoice: string (nullable = true)
```

- Final row count.

05:33 PM (1s) 42

```
# • Display the final Silver table row count.
silver_df.count()
```

[See performance \(1\)](#) [Checked for improvements](#)

524878

Part H: SQL Analysis on Silver Table

Task 15: Country-wise Revenue



▶ ▼ ✓ 05:33 PM (2s)



```
%sql
-- Sort the result by highest revenue.
select country, round(sum(total_amount),2) as total_revenue
from ordl.silver.online_retail
group by Country
order by total_revenue desc;
```

> [See performance \(1\)](#)

Table ▼ +

	country	total_revenue
13	PORTUGAL	33003.03
14	FINLAND	22546.08
15	SINGAPORE	21279.29
16	CHANNEL ISLANDS	20440.54
17	DENMARK	18955.34
18	ITALY	17483.24
19	HONG KONG	15483
20	CYPRUS	13502.85
21	AUSTRIA	10198.68
22	ISRAEL	8129.41
23	POLAND	7334.65
24	GREECE	4760.52
25	UNSPECIFIED	4740.94
26	ICELAND	4310
27	CANADA	3666.38

Task 16: Top 10 Products by Revenue

05:33 PM (1s)

```
%sql
-- Sort the result by highest revenue and show only the top 10 products.
select Description as product_name,
round(sum(total_amount),2) as total_revenue from ordl.silver.online_retail
group by Description
order by total_revenue desc
limit 10;
```

See performance (1)

Table +

	product_name	1.2 total_revenue
1	DOTCOM POSTAGE	206248.77
2	REGENCY CAKESTAND 3 TIER	174156.54
3	PAPER CRAFT , LITTLE BIRDIE	168469.6
4	WHITE HANGING HEART T-LIGHT HOLDER	106236.72
5	PARTY BUNTING	99445.23
6	JUMBO BAG RED RETROSPOT	94159.81
7	MEDIUM CERAMIC TOP STORAGE JAR	81700.92
8	POSTAGE	78101.88
9	Manual	77752.82
10	RABBIT NIGHT LIGHT	66870.03

Task 17: Top 10 Products by Quantity Sold

05:33 PM (2s)

```
%sql
-- Sort the result by highest quantity sold and show only the top 10 products.
select Description as product_name,
sum(Quantity) as total_quantity_sold from ordl.silver.online_retail
group by Description
order by total_quantity_sold desc
limit 10;
```

See performance (1)

Table +

	^A _C product_name	¹ ₃ total_quantity_sold
1	PAPER CRAFT , LITTLE BIRDIE	80995
2	MEDIUM CERAMIC TOP STORAGE JAR	78033
3	WORLD WAR 2 GLIDERS ASSTD DESIGNS	54951
4	JUMBO BAG RED RETROSPOT	48371
5	WHITE HANGING HEART T-LIGHT HOLDER	37872
6	POPCORN HOLDER	36749
7	PACK OF 72 RETROSPOT CAKE CASES	36396
8	ASSORTED COLOUR BIRD ORNAMENT	36362
9	RABBIT NIGHT LIGHT	30739
10	MINI PAINT SET VINTAGE	26633

10 rows | 1.72s runtime

Task 18: Monthly Revenue Trend

▶ ✓ 05:33 PM (2s)

```
%sql
-- Sort the result by month.
select invoice_month, round(sum(total_amount),2) as monthly_revenue
from ordl.silver.online_retail
group by invoice_month
order by monthly_revenue DESC;
```

>  See performance (1)

Table ▼ +

	^A _C invoice_month	1.2 monthly_revenue
1	2011-11	1503866.78
2	2011-10	1151263.73
3	2011-09	1056435.19
4	2010-12	821452.73
5	2011-05	769296.61
6	2011-06	760547.01
7	2011-08	757841.38
8	2011-07	718076.12
9	2011-03	716215.26
10	2011-01	689811.61
11	2011-12	637790.33
12	2011-04	536968.49
13	2011-02	522545.56

Task 19: Top 10 Customers by Revenue

05:33 PM (2s)52

```
%sql
-- Exclude rows where CustomerID is null. Sort by highest revenue and show only the top 10 customers.
select CustomerID, round(sum(total_amount),2) as customer_revenue from ordl.silver.online_retail
where CustomerID is not null
group by CustomerID
order by customer_revenue desc
limit 10;
```

> [See performance \(1\)](#)

Table ▾ +

	1.2 CustomerID	1.2 customer_revenue
1	14646	280206.02
2	18102	259657.3
3	17450	194390.79
4	16446	168472.5
5	14911	143711.17
6	12415	124914.53
7	14156	117210.08
8	17511	91062.38
9	16029	80850.84
10	12346	77183.6

Part I: Delta Time Travel

Task 20: Create a New Version of the Silver Table

Learner-written code.

Updated Silver table preview.

05:33 PM (3)

56

• Overwrite the Silver Delta path.
filtered_df.write\
 .format("delta")\
 .mode("overwrite")\
 .option("overwriteSchema", True)\
 .saveAsTable(f'{catalog}.{silver_schema}.{data_source}')

> [See performance \(1\)](#) ✓ Checked for improvements

05:33 PM (1)

57

• Preview the updated table.
updated_df = spark.table(f'{catalog}.{silver_schema}.{data_source}')
display(updated_df.limit(10))

> [See performance \(1\)](#) ✓ Checked for improvements

> updated_df: pyspark.sql.connect.dataframe.DataFrame = [InvoiceNo: string, StockCode: string ... 10 more fields]

Table +

Q F I

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	invoice
1	536387	22466	FAIRY TALE COTTAGE NIGHTLIGHT	432	2010-12-01T09:58:00.000+00:00	1.45	16029	UNITED KINGDOM	2010-12-
2	536390	85123A	WHITE HANGING HEART T-LIGHT HOLDER	64	2010-12-01T10:19:00.000+00:00	2.55	17511	UNITED KINGDOM	2010-12-
3	536394	22632	HAND WARMER RED RETROSPOT	96	2010-12-01T10:39:00.000+00:00	1.85	13408	UNITED KINGDOM	2010-12-
4	536536	84879	ASSORTED COLOUR BIRD ORNAMENT	80	2010-12-01T13:45:00.000+00:00	1.69	18144	UNITED KINGDOM	2010-12-
5	536544	DOT	DOTCOM POSTAGE	1	2010-12-01T14:32:00.000+00:00	569.77	null	UNITED KINGDOM	2010-12-
6	536575	22095	LADS ONLY TISSUE BOX	252	2010-12-01T16:01:00.000+00:00	1.25	13777	UNITED KINGDOM	2010-12-
7	536586	22848	BREAD BIN DINER STYLE PINK	8	2010-12-01T16:25:00.000+00:00	14.95	18229	UNITED KINGDOM	2010-12-
8	536618	22943	CHRISTMAS LIGHTS 10 VINTAGE BAUBLES	24	2010-12-02T10:17:00.000+00:00	4.25	17017	UNITED KINGDOM	2010-12-

Updated row count.

05:33 PM (1)

58

• Display the updated row count.
updated_df.count()

> [See performance \(1\)](#) ✓ Checked for improvements

13547

Learner-written command.

Delta table history output.

Task 21: View Delta Table History

05:33 PM (4)

60

%sql
-- Write and execute the command to view the history of the Silver Delta table. The output should show multiple versions of the Silver Delta table.
DESCRIBE HISTORY ordl.silver.online_retail;

> [See performance \(1\)](#) ✓ Checked for improvements

Table +

Q F I

	version	timestamp	userid	userName	operation	operationParameters
1	13	2026-06-13T12:05:05.000+00:00	758515865392...	analyst.kraj@gmail.co...	CREATE OR REPLACE TABLE AS SELECT	> {"isV1SaveAsTableOverwrite":"true","partitionBy":[""],"clusterBy":[""],"description":"nu
2	12	2026-06-13T12:04:47.000+00:00	758515865392...	analyst.kraj@gmail.co...	CREATE OR REPLACE TABLE AS SELECT	> {"isV1SaveAsTableOverwrite":"true","partitionBy":[""],"clusterBy":[""],"description":"nu
3	11	2026-06-12T06:35:05.000+00:00	758515865392...	analyst.kraj@gmail.co...	CREATE OR REPLACE TABLE AS SELECT	> {"isV1SaveAsTableOverwrite":"true","partitionBy":[""],"clusterBy":[""],"description":"nu
4	10	2026-06-12T06:34:51.000+00:00	758515865392...	analyst.kraj@gmail.co...	CREATE OR REPLACE TABLE AS SELECT	> {"isV1SaveAsTableOverwrite":"true","partitionBy":[""],"clusterBy":[""],"description":"nu
5	9	2026-06-12T06:06:15.000+00:00	758515865392...	analyst.kraj@gmail.co...	CREATE OR REPLACE TABLE AS SELECT	> {"isV1SaveAsTableOverwrite":"true","partitionBy":[""],"clusterBy":[""],"description":"nu
6	8	2026-06-12T06:05:59.000+00:00	758515865392...	analyst.kraj@gmail.co...	CREATE OR REPLACE TABLE AS SELECT	> {"isV1SaveAsTableOverwrite":"true","partitionBy":[""],"clusterBy":[""],"description":"nu
7	7	2026-06-11T17:19:56.000+00:00	758515865392...	analyst.kraj@gmail.co...	CREATE OR REPLACE TABLE AS SELECT	> {"isV1SaveAsTableOverwrite":"true","partitionBy":[""],"clusterBy":[""],"description":"nu
8	6	2026-06-11T16:58:14.000+00:00	758515865392...	analyst.kraj@gmail.co...	CREATE OR REPLACE TABLE AS SELECT	> {"isV1SaveAsTableOverwrite":"true","partitionBy":[""],"clusterBy":[""],"description":"nu
9	5	2026-06-11T16:41:47.000+00:00	758515865392...	analyst.kraj@gmail.co...	CREATE OR REPLACE TABLE AS SELECT	> {"isV1SaveAsTableOverwrite":"true","partitionBy":[""],"clusterBy":[""],"description":"nu
10	4	2026-06-11T16:39:41.000+00:00	758515865392...	analyst.kraj@gmail.co...	CREATE OR REPLACE TABLE AS SELECT	> {"isV1SaveAsTableOverwrite":"true","partitionBy":[""],"clusterBy":[""],"description":"nu
11	3	2026-06-11T15:45:34.000+00:00	758515865392...	analyst.kraj@gmail.co...	CREATE OR REPLACE TABLE AS SELECT	> {"isV1SaveAsTableOverwrite":"true","partitionBy":[""],"clusterBy":[""],"description":"nu
12	2	2026-06-11T15:17:28.000+00:00	758515865392...	analyst.kraj@gmail.co...	CREATE OR REPLACE TABLE AS SELECT	> {"isV1SaveAsTableOverwrite":"true","partitionBy":[""],"clusterBy":[""],"description":"nu
13	1	2026-06-11T13:24:05.000+00:00	758515865392...	analyst.kraj@gmail.co...	CREATE OR REPLACE TABLE AS SELECT	> {"isV1SaveAsTableOverwrite":"true","partitionBy":[""],"clusterBy":[""],"description":"nu

Task 22: Read Older Version Using Delta Time Travel

Learner-written time travel code.

Older version output.

05:33 PM (2s) 62

```
# • Preview the older version
version_0_df = spark.read\
    .format("delta")\
    .option("versionAsOf", 0)\
    .table(f'{catalog}.{silver_schema}.{data_source}')
display(version_0_df.limit(5))
```

See performance (1) ✓ Checked for improvements

version_0_df: pyspark.sql.connect.dataframe.DataFrame = [InvoiceNo: string, StockCode: string ... 10 more fields]

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	InvoiceDate
1	536367	48187	DOORMAT NEW ENGLAND	4	2010-12-01T08:34:00.000+00:00	7.95	13047	UNITED KINGDOM	2010-12-01
2	536373	22752	SET 7 BABUSHKA NESTING BOXES	2	2010-12-01T09:02:00.000+00:00	7.65	17850	UNITED KINGDOM	2010-12-01
3	536375	37370	RETRO COFFEE MUGS ASSORTED	6	2010-12-01T09:32:00.000+00:00	1.06	17850	UNITED KINGDOM	2010-12-01
4	536380	22961	JAM MAKING SET PRINTED	24	2010-12-01T09:41:00.000+00:00	1.45	17809	UNITED KINGDOM	2010-12-01

5 rows | 2.14s runtime Refreshed 1 hour ago

Latest version output.

05:33 PM (1s) 63

```
# • Preview the latest version.
latest_df = spark.read\
    .format("delta")\
    .table(f'{catalog}.{silver_schema}.{data_source}')
display(latest_df.limit(5))
```

See performance (1) ✓ Checked for improvements

latest_df: pyspark.sql.connect.dataframe.DataFrame = [InvoiceNo: string, StockCode: string ... 10 more fields]

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	InvoiceDate
1	536387	22466	FAIRY TALE COTTAGE NIGHTLIGHT	432	2010-12-01T09:58:00.000+00:00	1.45	16029	UNITED KINGDOM	2010-12-01
2	536390	85123A	WHITE HANGING HEART T-LIGHT HOLDER	64	2010-12-01T10:19:00.000+00:00	2.55	17511	UNITED KINGDOM	2010-12-01
3	536394	22632	HAND WARMER RED RETROSPOT	96	2010-12-01T10:39:00.000+00:00	1.85	13408	UNITED KINGDOM	2010-12-01
4	536536	84879	ASSORTED COLOUR BIRD ORNAMENT	80	2010-12-01T11:34:00.000+00:00	1.69	18144	UNITED KINGDOM	2010-12-01

5 rows | 1.33s runtime Refreshed 1 hour ago

Row count comparison.

05:33 PM (1s) 64

```
# • Show row count of the older version
print(f'row count of the older version: {version_0_df.count()}')

# • Show row count of the latest version.
print(f'row count of the latest version: {latest_df.count()}')
```

See performance (2) ✓ Checked for improvements

row count of the older version: 524878
row count of the latest version: 13547

Part J: VACUUM

Screenshot showing successful VACUUM execution, or screenshot showing the attempted VACUUM command and system message/error

Part J: VACUUM

Task 23: Run or Attempt VACUUM

Last execution failed

66

1

Write and execute the VACUUM command for the Silver Delta table. Retain data for 168 hours. Run the command only if your Databricks environment allows it. If the environment blocks the command, capture the error/output.

2

spark.sql(f'VACUUM {catalog}.{silver_schema}.{data_source} RETAIN 168 HOURS')

>

[See performance \(1\)](#)

✓ Checked for improvements

❌

> SyntaxError: unterminated f-string literal (detected at line 2) (command-8793481462467146-1609787172, line 2)

wsfs/fuse/wsfs.go:964 Error(message=Cannot find child enum-stubs.pyi, errno=no such file or directory, statusCode=0, stack=)

[Trace ID: 00-7ff3f6a5830b47ea9395e4076151fe4d-a7a99077469aaa26-00]

⚡ Diagnose error

🔍 Debug

Genie Code Quick Fix: ON